

UNIVERSITÀ POLITECNICA DELLE MARCHE
INGEGNERIA INFORMATICA E DELL'AUTOMAZIONE

Controllo di trazione e sterzata di un veicolo in scala 1:10 mediante radiocomando



Corso di
LABORATORIO DI AUTOMAZIONE
Anno accademico 2025-2026

Studenti:

Tarek Naja
Marco Sambughi
Sara Vaccaro

Professore:

Andrea Bonci

Dottorando:

Alessandro Di Biase



Dipartimento di Ingegneria dell'Informazione

Indice

Introduzione	1
1 Descrizione del sistema	3
1.1 Controllo veicolo	4
1.1.1 Controllo della trazione	4
1.1.2 Controllo dello sterzo	5
2 Hardware	7
2.1 Telaio e meccaniche di supporto	7
2.2 Nucleo-144 STM32H755zi-q	8
2.3 Motore DC Reely 550er	8
2.4 Driver Motore Cytron MD10C R3	9
2.5 Encoder incrementale CUI Devices AMT102	10
2.6 Servomotore Miuzei MS24	10
2.7 Schema dei collegamenti	11
3 Software	13
3.1 Diagramma di flusso	13
3.2 Gestione singoli componenti	14
3.2.1 Gestione componente1	15
3.2.2 Gestione componente2	15
3.3 Funzionamento complessivo	15
4 Test e risultati	16
4.1 Test1	16
4.2 Test2	16
Conclusioni e sviluppi futuri	17
Appendici	17
A Appendice1	18

Introduzione

Spiegazione dettagliata del task assegnato al gruppo (cosa avete fatto e su quale sistema). Se per lo svolgimento del task è stato necessario interagire con altri gruppi che hanno lavorato sullo stesso sistema presentare anche brevemente i task svolti dagli altri gruppi e spiegare la loro interconnessione. Se si è invece optato per far confluire il lavoro di più gruppi in un'unica relazione, va spiegato qual era il task iniziale dei singoli gruppi e poi come è avvenuta l'integrazione.

In sintesi, l'introduzione deve permettere al lettore di capire subito su cosa avete lavorato. Al termine dell'introduzione inserite anche una breve descrizione per capitoli del contenuto della restante parte della relazione (es Nel Capitolo 2 viene descritto...).

Nel seguito la struttura in capitoli di massima per ogni relazione, da adattare qualora necessario.

NOTA: La relazione va pensata e scritta nell'ottica di dare a chi verrà dopo di voi un documento utile per capire a che punto è arrivata l'attività svolta dal vostro gruppo e per metterli in condizione di portare avanti il vostro lavoro. Di conseguenza dovete essere chiari e diretti, riportando solo le informazioni necessarie e evidenziando in modo critico cosa effettivamente funziona correttamente e cosa va migliorato

1 Descrizione del sistema

La seguente relazione si basa sull'analisi dell'architettura di comunicazione via radio tra radiocomando e microcontrollore. Nel dettaglio, il trasmettitore impiega onde elettromagnetiche per l'invio dei pacchetti dati, sfruttando la modulazione di un'onda portante che codifica le informazioni variando parametri dell'onda stessa, quali l'ampiezza, la frequenza o la fase. L'espressione matematica dell'onda portante è definita dalla seguente funzione:

$$s(t) = A \cos(2\pi f_c t + \phi) \quad (1.1)$$

dove:

- A : ampiezza dell'onda;
- f_c : frequenza portante;
- t : tempo;
- ϕ : fase iniziale.

L'azione dell'utente sui comandi del radiocomando genera una sequenza di bit (segnale elettrico), che l'antenna del trasmettitore provvede a convertire in onda elettromagnetica, irradiandola nello spazio a una determinata frequenza f . Il modulo ricevitore, essendo sintonizzato sulla stessa frequenza, intercetta il segnale e lo riconverte nel formato elettrico. Questi dati vengono poi inoltrati al microcontrollore, il quale ha il compito di interpretare le istruzioni e comandare l'hardware per eseguire determinate azioni. Per la traduzione pratica dei comandi, il microcontrollore impiega la modulazione a larghezza di impulso (PWM). Il parametro fondamentale per questa tecnica è il *Duty Cycle*, calcolato come segue:

$$\text{Duty Cycle (\%)} = \frac{T_{on}}{T_{on} + T_{off}} \times 100 \quad (1.2)$$

dove T_{on} è la durata dell'impulso "alto", T_{off} la durata dell'impulso "basso" e la loro somma definisce il periodo totale dell'onda. Per quanto riguarda l'analisi della traiettoria del veicolo, si adotta il modello cinematico della bicicletta, approssimando la dinamica del veicolo a quattro ruote riducendola a due. Le equazioni cinematiche descrivono il moto attraverso le variabili di posizione (x, y) e l'angolo di orientamento della vettura. Considerando costante la velocità v del veicolo, l'assenza di slittamento e che l'intera rotazione del veicolo avvenga attorno al CIR (Centro Istantaneo di Rotazione), il modello viene descritto dal seguente sistema:

$$\begin{cases} \dot{x}(t) = v \cos(\theta) \\ \dot{y}(t) = v \sin(\theta) \\ \dot{\theta}(t) = \frac{v}{L} \tan(\delta) \end{cases} \quad (1.3)$$

I parametri fondamentali del modello sono:

- L : distanza tra asse anteriore e posteriore (passo);
- δ : angolo di sterzata (ruote anteriori);
- θ : angolo di imbardata del veicolo;
- v : velocità della macchina;
- $\dot{x}, \dot{y}$: velocità nei due assi (derivate della posizione rispetto al tempo).

Nel caso in cui si considerasse lo slittamento si passerebbe ad un modello dinamico e la velocità di rotazione non dipenderebbe più solo dalla geometria. Il modello descritto quindi è una semplificazione del modello reale, ma risulta uno strumento efficace per comprendere la cinematica di base del veicolo e la traiettoria in funzione dell'angolo di sterzo, sfruttando unicamente due assi.

1.1 Controllo veicolo

Nelle prossime sezioni verranno analizzate, dal punto di vista teorico, le tecniche di controllo utilizzate per la gestione degli attuatori presenti sul veicolo.

1.1.1 Controllo della trazione

Per il controllo della trazione, ovvero la regolazione della velocità longitudinale del veicolo, si è optato per un sistema di controllo in retroazione (*closed-loop*). Tale architettura risulta fondamentale non solo per garantire il preciso inseguimento della velocità di riferimento, ma anche per compensare i disturbi esterni non prevedibili, quali le variazioni di attrito del terreno, le pendenze o le cadute di tensione della batteria durante l'erogazione di corrente. Il sistema misura costantemente la velocità reale del veicolo e la confronta con la velocità di riferimento, generando un segnale di errore $e(t)$ da minimizzare:

$$e(t) = v_{\text{target}} - v_{\text{reale}} \quad (1.4)$$

Il feedback di velocità è fornito da un encoder incrementale calettato sull'albero motore: il sensore genera una serie di impulsi che vengono conteggiati dal microcontrollore all'interno di un periodo di campionamento a tempo fisso. Dalla derivata discreta di questi conteggi si ricava l'effettiva velocità di rotazione del rotore. Per minimizzare l'errore $e(t)$ e calcolare lo sforzo di controllo da applicare al motore, tradotto poi nel *Duty Cycle* del segnale PWM, viene impiegato un controllore PID (Proporzionale-Integrale-Derivativo). La legge di controllo nel dominio del tempo continuo è descritta dalla seguente equazione:

$$u(t) = K_p \cdot e(t) + K_i \int e(t) dt + K_d \frac{de(t)}{dt} \quad (1.5)$$

dove $u(t)$ rappresenta l'azione di comando generata dal controllore, mentre K_p , K_i e K_d sono rispettivamente le costanti di guadagno dell'azione proporzionale, integrale e derivativa. Nello specifico:

- **Guadagno proporzionale (K_p):** fornisce un'uscita di controllo direttamente proporzionale all'errore istantaneo. Aumentare il guadagno K_p rende il sistema più reattivo e riduce il tempo di salita. Tuttavia, da sola non è sufficiente a garantire il perfetto inseguimento del riferimento: un'azione puramente proporzionale lascia un errore a regime e, se eccessiva, induce forti oscillazioni e instabilità.
- **Guadagno integrale (K_i):** somma l'errore nel tempo, tenendo conto della “storia” del sistema. Il suo scopo fondamentale è annullare completamente l'errore a regime. Tuttavia, un accumulo prolungato di questo termine, ad esempio durante la fase di avvio o nel caso in cui le ruote slittino senza far muovere il veicolo, porta a un eccessivo incremento dell'azione di controllo. Questo si traduce in partenze improvvise o risposte sproporzionate non appena il target cambia, problematica che nel software è stata mitigata azzerando il termine integrale in specifiche condizioni di sicurezza.
- **Guadagno derivativo (K_d):** reagisce alla velocità di variazione dell'errore, anticipandone l'andamento futuro. Funge essenzialmente da “smorzatore”, con lo scopo di contrastare le eventuali oscillazioni introdotte dalle componenti precedenti. Tuttavia, per il controllo longitudinale di veicoli di questa scala, l'azione derivativa risulta spesso superflua o persino controproducente, in quanto tende ad amplificare matematicamente le normali micro-vibrazioni meccaniche o i piccoli scatti letti dall'encoder. Per questo motivo, è prassi comune nella robotica mobile di base porre direttamente $K_d = 0$, semplificando l'architettura a un più robusto e stabile controllore PI.

La determinazione dei parametri ottimali (K_p , K_i , K_d) prende il nome di calibrazione o *tuning*. Per la dinamica del veicolo in esame, tale procedura è stata condotta mediante calibrazione empirica in laboratorio. Partendo da valori conservativi e analizzando visivamente e tramite telemetria la risposta del motore alle variazioni di velocità richieste dal radiocomando, i guadagni sono stati ottimizzati per garantire l'inseguimento del target senza innescare instabilità.

1.1.2 Controllo dello sterzo

La gestione direzionale del veicolo è affidata al sistema di sterzo delle ruote anteriori e ai segnali inviati dal radiocomando. L'attuazione meccanica è realizzata mediante un servomotore, un tipo particolare di motore che, agendo sui braccetti dello sterzo, è in grado di ruotare in una specifica posizione e mantenerla. Inoltre, in questo prototipo è stato implementato un controllo a ciclo chiuso per garantire la massima precisione nell'inseguimento della traiettoria e compensare i disturbi dinamici come gli attriti del terreno. Di conseguenza, il controllo prevede l'utilizzo di un regolatore PID, implementato all'interno del microcontrollore STM32. Il controllore riceve in ingresso l'errore $e(t)$, definito come la differenza tra l'angolo di sterzo di riferimento richiesto $\delta_{\text{ref}}(t)$, letto dal radiocomando, e l'angolo reale misurato dai sensori di bordo $\delta_{\text{reale}}(t)$:

$$e(t) = \delta_{\text{ref}}(t) - \delta_{\text{reale}}(t) \quad (1.6)$$

L'azione correttiva calcolata dall'algoritmo PID viene tradotta in un segnale di pilotaggio PWM inviato al servomotore. Per questa tipologia di attuatori, il controllo della posizione

non dipende dal *Duty Cycle* percentuale classico, bensì dalla larghezza assoluta dell'impulso positivo T_{on} all'interno di un periodo fisso di 20 ms, corrispondente ad una frequenza di 50 Hz.

2 Hardware

Nel presente capitolo viene analizzata in dettaglio l'architettura hardware del prototipo, descrivendo le specifiche tecniche e il ruolo funzionale di ciascun componente. Verranno presi in analisi i seguenti elementi costitutivi:

- Telaio e meccanica di supporto;
- Microcontrollore Nucleo-144 STM32H755zi-q;
- Motore DC Reely 550er;
- Driver Motore Cytron MD10C R3;
- Encoder incrementale CUI Devices AMT102;
- Servomotore Miuzei MS24;
- Sistema di Alimentazione e Power Distribution Board (PDB);
- Batteria ai polimeri di litio (LiPo) 2S (2 celle);
- Unità di Inerzia Magneto-Inerziale (IMU) Bosch BNO055;
- Radiocomando e Ricevitore Microzone.

2.1 Telaio e meccaniche di supporto

La struttura portante e la cinematica del veicolo sono basate su uno chassis in scala 1:10 derivato dal settore del modellismo dinamico. Per rendere lo chassis idoneo agli scopi del progetto, la struttura originale è stata sottoposta a una serie di interventi di personalizzazione. In particolare, è stata progettata e installata una piastra di supporto superiore per incrementare la superficie utile a bordo, permettendo l'alloggiamento di tutta la componentistica elettronica.



Figure 2.1

2.2 Nucleo-144 STM32H755zi-q

L'unità di elaborazione centrale e di controllo del veicolo è costituita dalla scheda di sviluppo Nucleo-144 dotata del microcontrollore ad altissime prestazioni STM32H755ZIT6Q di STMicroelectronics. L'elemento distintivo di questo dispositivo è la sua architettura hardware *dual-core* eterogenea, che mette a disposizione due processori indipendenti a bordo dello stesso chip:

- **Core ARM Cortex-M7 (32-bit):** operante a una frequenza massima di 480 MHz.
- **Core ARM Cortex-M4 (32-bit):** operante a una frequenza massima di 240 MHz.

L'interazione e lo scambio di dati ad alta velocità tra i due core avvengono in modo sincrono tramite un'interfaccia di comunicazione interna basata su memoria condivisa e semafori hardware. La scheda offre un ricco set di periferiche hardware dedicate che risultano fondamentali per gli obiettivi del progetto: i timer avanzati per la generazione dei segnali PWM di trazione e sterzo, i timer configurati in modalità *Encoder Interface* per la lettura diretta in quadratura dell'encoder AMT102, e i bus di comunicazione analogici e digitali, come I²C o UART, per l'interfacciamento con i sensori di bordo. La configurazione delle periferiche e lo sviluppo del codice in linguaggio C sono stati interamente gestiti tramite l'ecosistema software ufficiale di STMicroelectronics, utilizzando lo strumento di generazione grafica STM32CubeMX per l'inizializzazione dell'hardware e l'ambiente di sviluppo integrato STM32CubeIDE per la scrittura, compilazione e il *debug* del *firmware*.

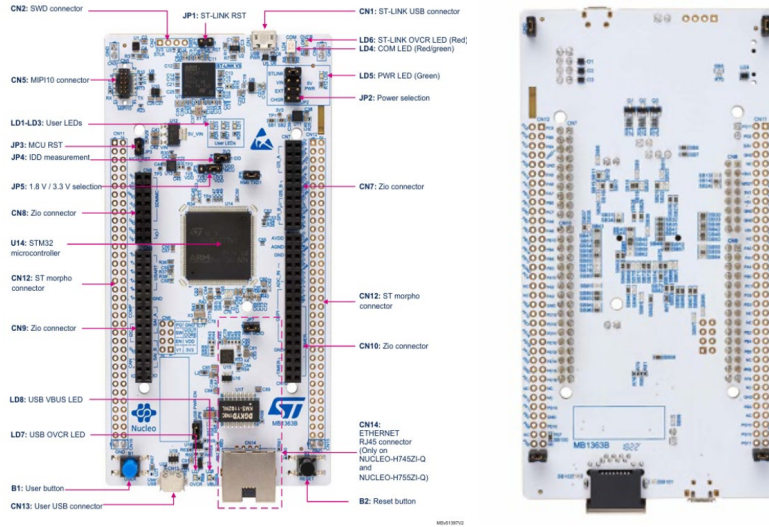


Figure 2.2

2.3 Motore DC Reely 550er

La trazione del veicolo viene garantita da un motore collegato alla trasmissione dello chassis.



Figure 2.3

In particolare, viene utilizzato un motore DC a collettore con le seguenti caratteristiche:

Parametro	Valore
Tensione nominale	4.8 – 12 VDC
Velocità di rotazione	1000 – 18000 RPM
Coppia	100 – 900 g·cm
Diametro albero	3.175 mm

2.4 Driver Motore Cytron MD10C R3

Il motore viene controllato tramite modulazione PWM attraverso il driver MD10C-R3. In particolare, il controllo del motore attraverso questo driver avviene tramite due segnali:

- **DIR:** controllo del verso di rotazione (Segnale digitale alto/basso)
- **PWM:** controllo della velocità di rotazione (segnale PWM)



Figure 2.4

2.5 Encoder incrementale CUI Devices AMT102

Inoltre, per effettuare il controllo del motore in velocità è necessario avere un feedback sulla velocità del motore stesso: ciò è stato reso possibile montando un encoder sull'asse posteriore del motore. In particolare, è stato utilizzato un encoder rotativo incrementale modello AMT102.



Figure 2.5

2.6 Servomotore Miuzei MS24

Per operare il meccanismo di sterzo del veicolo è stato impiegato un servo motore: questo dispositivo riesce a regolare il movimento angolare in modo molto preciso e proprio per questo viene comunemente utilizzato in diverse apparecchiature elettroniche e applicazioni che richiedono un controllo di posizione accurato.



Figure 2.6

2.7 Schema dei collegamenti

Inserire uno schema dei collegamenti analogo a quello in Figura 2.7. Si suggerisce per lo scopo di usare il software "draw.io". Nello schema riportare tutti i collegamenti tra i componenti, specificando quale pin di un componente1 si collega con quale pin di un componente2 (per la maggior parte dei collegamenti dovete cioè avere una linea con 2 label). Inserire anche una descrizione testuale dello schema.

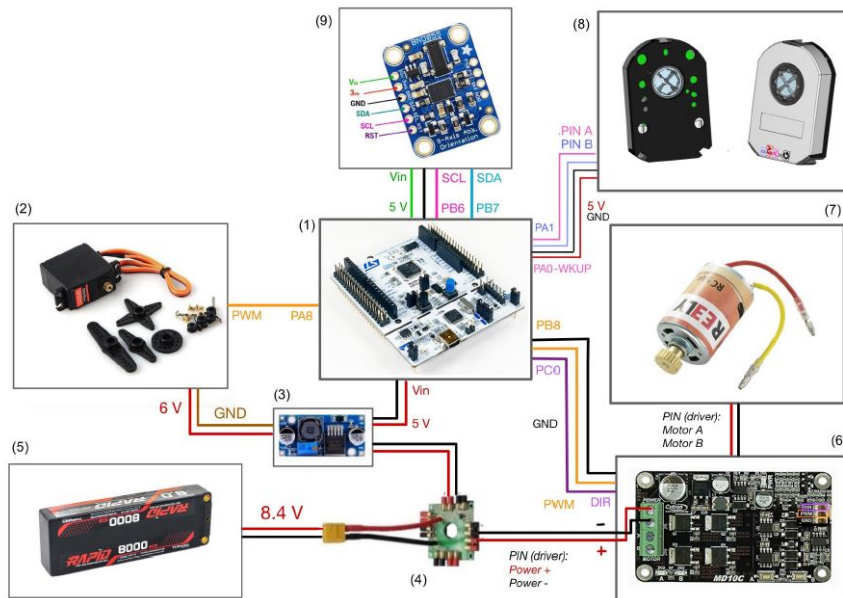


Figure 2.7: *Schema dei collegamenti*

3 Software

Nell'introduzione al capitolo specificate anche la versione dell'STM32CubeIDE e dell'STM32CubeMX che avete usato (se avete cambiato versione nel corso del progetto mettete l'ultima, quella su cui è sviluppato il codice che consegnate).

3.1 Diagramma di flusso

Inserire un diagramma di flusso analogo a quello in Figura 3.1 che spiega il funzionamento generale del codice sviluppato. Inserire anche una descrizione testuale del funzionamento.

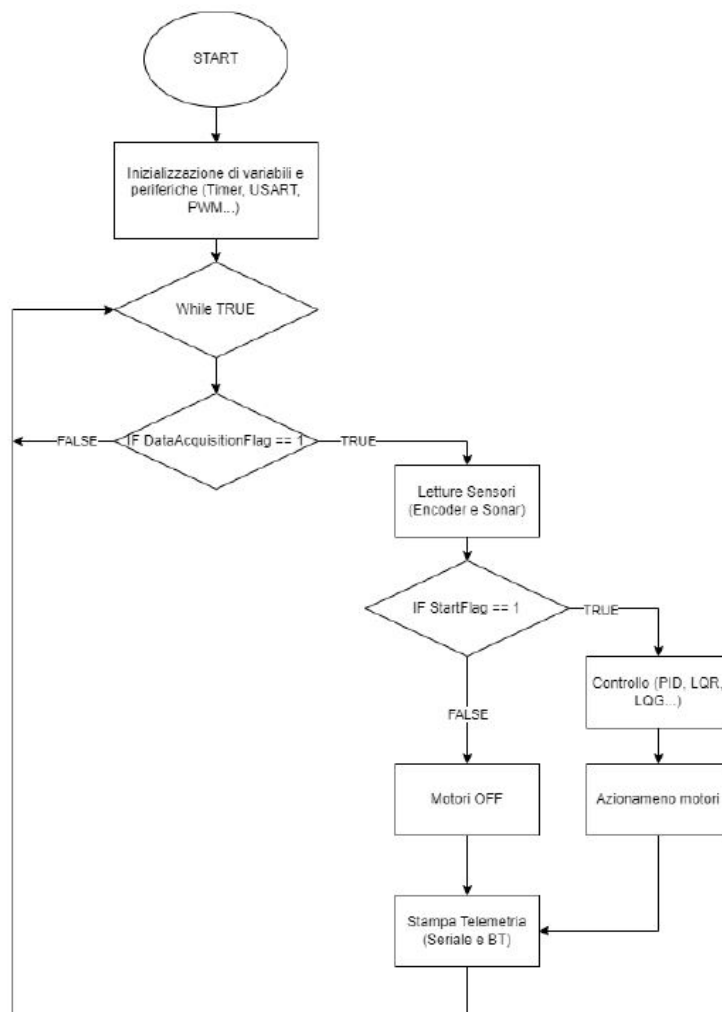


Figure 3.1: *Diagramma di flusso del codice sviluppato*

3.2 Gestione singoli componenti

Dedicare un paragrafo ad ogni componente in cui viene spiegato il codice che lo gestisce. Inserite prima una descrizione generale di come si gestisce un componente di quel tipo (aiutandovi con schemi o altro). Poi le configurazioni che sono state fatte sul .ioc per poterlo utilizzare (quale periferiche sono state abilitate, come sono state configurate.. mettete degli screen del .ioc). Infine inserite e spiegate le parti di codice che lo gestiscono.

NOTA per il codice: per rendere il codice modulare e riutilizzabile è buona prassi non caricare troppo il main ma creare una libreria per ogni componente. Quindi il componente1 avrà un header file "componente1.h" e un source file "componente1.c", nel source file sono implementate le funzioni che gestiscono il componente, nell'header file ci sono i prototipi di tali funzioni in modo che esse possano essere usate nel main includendolo.

NOTA per il codice: non inserire mai nel codice dei parametri numerici senza contesto ma renderli delle costanti definite (usando la direttiva %define). Se sono delle costanti relative ad uno specifico componente vanno inserite nel relativo header file.

NOTA per il codice: per inserire il codice, anziché utilizzare screenshot utilizzate i seguenti comandi.

Per codice scritto in latex (con linguaggio Python):

Listing 3.1: "Codice in Python"

```
import numpy as np

def incmatrix(genl1, genl2):
    m = len(genl1) # The length of the first array
    n = len(genl2) # The length of the second array
    sum = 0

    # Compute
    for i in range(0, n):
        for j in range(0, m):
            sum += genl1[n]*genl2[m]

    # Print
    print("The sum is %2d" %(sum))

    return M
```

Per codice richiamato da file (con linguaggio C):

```
1 void main(int n1){
2     int a = 1;          // The first number
3     int b = 2;          // The second number
4     return a+b*n1;      // The sum of the numbers
5 }
```

Listing 3.2: "Codice in C"

Funziona anche per codice MATLAB:

```
1 %% PREPARE WORKSPACE
2 close all
3 clearvars
4 clc
5
6 %% OPERATIONS
7 sayhello;
8
9 %% FUNCTIONS
10 function sayhello
11     fprintf("Hello world!");
12 end
```

Listing 3.3: "Codice in MATLAB"

3.2.1 Gestione componente1

3.2.2 Gestione componente2

3.3 Funzionamento complessivo

Dopo aver spiegato il codice che gestisce i singoli componenti, inserire e commentare le porzioni di codice relative al funzionamento complessivo del programma.

4 Test e risultati

Dedicare un capitolo a tutti i test effettuati con relativi risultati, includete sia i test finali relativi al funzionamento complessivo, sia i test delle singole parti (se significativi).

Descrivete in dettaglio le condizioni in cui sono stati svolti i test in modo che siano ripetibili.

Durante i test fate dei video e acquisite i dati (es usando la funzione di log di Putty), per i test più rilevanti è opportuno consegnare anche questo materiale per documentare i test effettuati.

Nella relazione riportate i risultati con dei grafici come quello in Figura 4.1 (inserite sempre nei grafici le label sugli assi con grandezza e relativa unità di misura). Commentate in modo critico i risultati ottenuti, evidenziando sia quelli positivi sia quelli negativi.

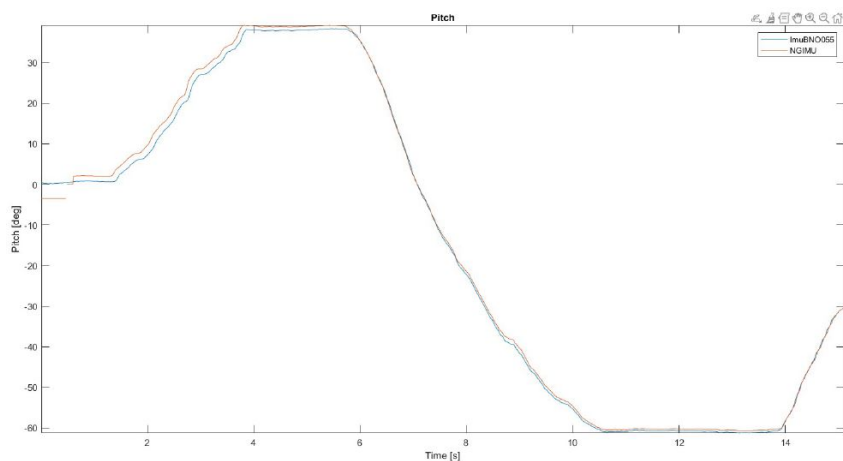


Figure 4.1: *Esempio di grafico*

4.1 Test1

Per ogni test riportate: cosa state testando, in che condizioni si è svolto il test, riferimenti a video/file di log che consegnate insieme a relazione e codice, grafici dei risultati, commento dei risultati.

4.2 Test2

Conclusioni e sviluppi futuri

Riassumere brevemente il lavoro svolto rispetto al task assegnato, evidenziando quali risultati sono stati raggiunti e quali no. Se ci sono aspetti del task non completati spiegare quali sono stati i problemi riscontrati in merito.

Inserire considerazioni personali su possibili sviluppi futuri dell'attività svolta (idee per migliorarla che non avete avuto modo di sperimentare, aspetti che suggerite di approfondire, problemi da risolvere..).

A Appendice1

Se necessario ricorrete alle appendici per spiegare le parti "di contorno" dell'attività svolta e/o ciò che non riuscite ad inserire nello schema generale dei capitoli della relazione (es acquisizione dei dati con Matlab).

Bibliografia